OPERATING SYSTEMS

LECTURE SLIDES ARE ADAPTED/MODIFIED FROM SLIDES PROVIDED BY THE TEXTBOOK, **COMPUTER SCIENCE: AN OVERVIEW** BY J. GLENN BROOKSHEAR AND DENNIS BRYLOW

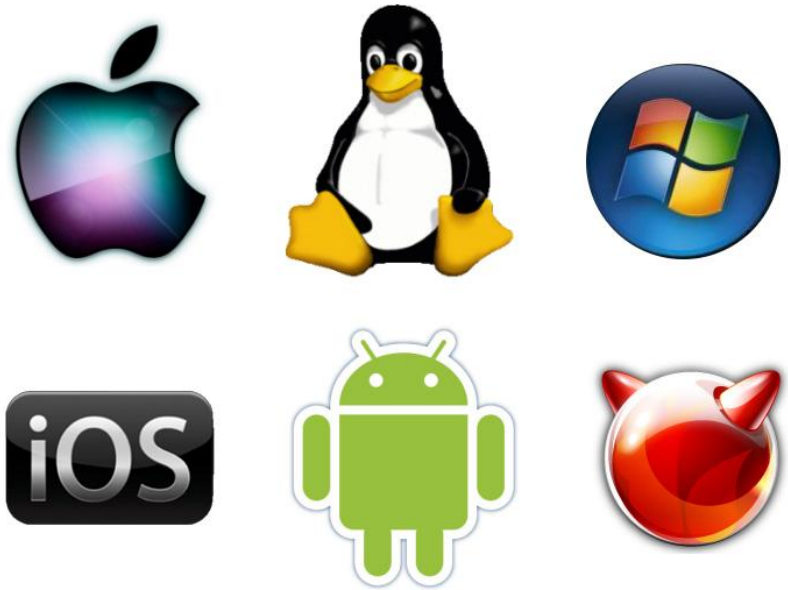
PUBLISHER PEARSON

Contents

- The History of Operating Systems
- Operating System Architecture
- Coordinating the Machine’s Activities
- Handling Competition Among Processes
- Security

OPERATING SYSTEMS - Definition

- An operating system (OS) is ***system software*** that acts as an ***intermediary*** between computer hardware and application software.
- **“Manager"** of your computer that coordinates everything happening inside.



Key roles of an OS

- Manages hardware resources (CPU, memory, storage, devices)
- Provides a user interface (GUI or command-line)
- Executes and manages applications
- Handles file systems and data storage
- Ensures security and access control

Why Do We Need an Operating System?

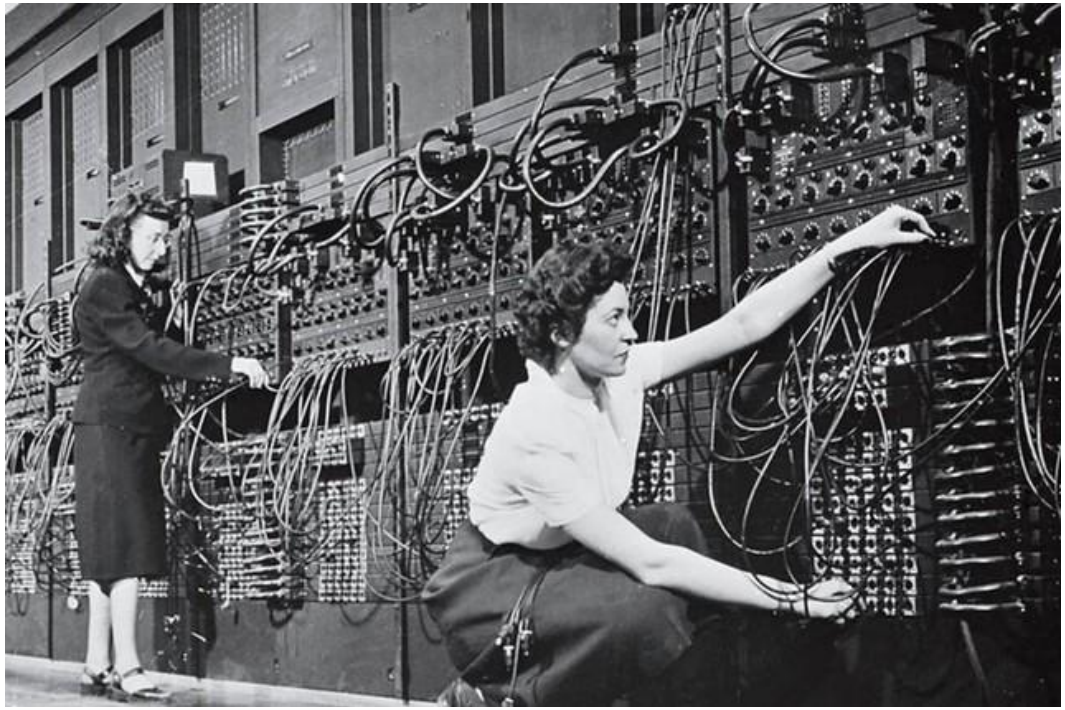
- If every program you wrote had to directly control the hardware—you'd need to write code to manage:
  - CPU, allocate memory, read from disk, and handle every device differently.
- This would be:
  - Extremely complex
  - Error-prone
  - Insecure
  - Inefficient
- The OS abstracts these complexities, providing simple interfaces (system calls) that programs can use without worrying about hardware details.

The First Generation (1945-1955)

- **No Operating System**
- Computers were room-sized machines built with vacuum tubes and plug boards.
  - Programmed by ***physically rewiring circuits***
  - Programmers had ***direct access*** to the hardware
  - Program ran at a time from start to finish
  - Programming meant setting switches and plugging cables
  - Jobs took hours or days to set up

The First Generation (1945-1955)

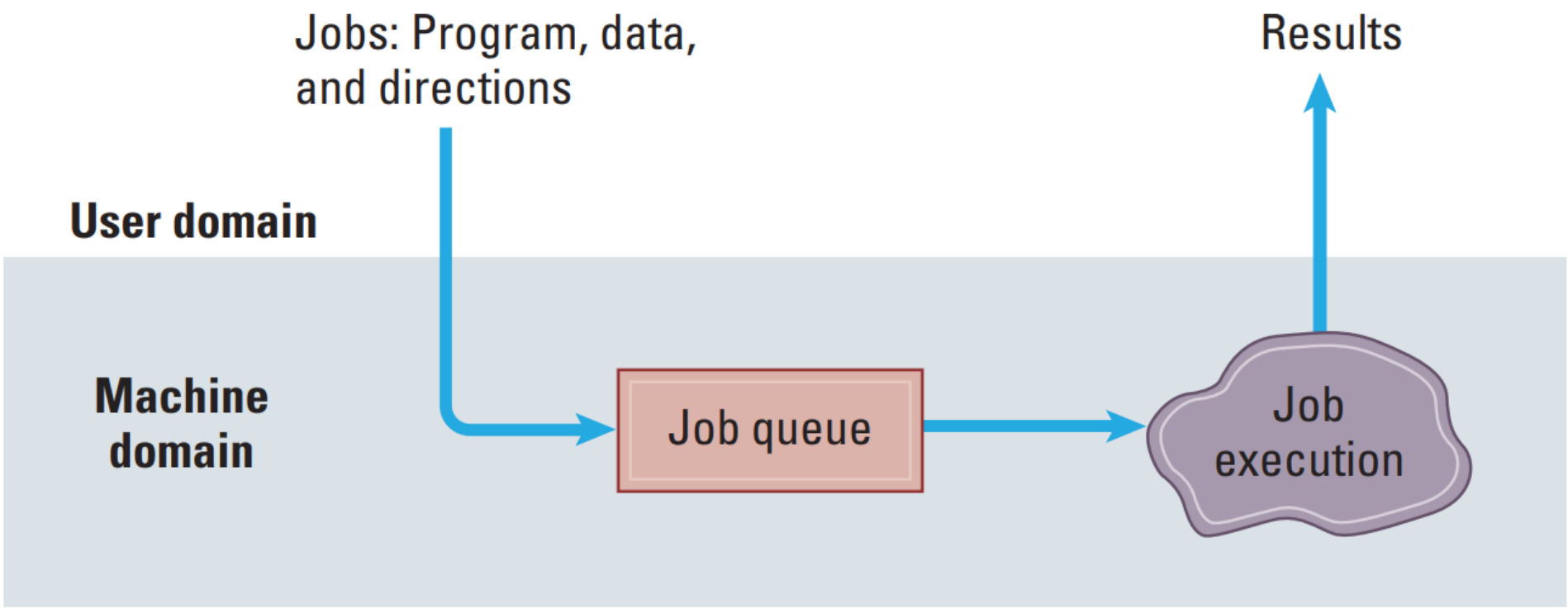
- **The Problem**: incredibly inefficient.
  - Most of the computer's time was spent idle while operators physically loaded programs and data



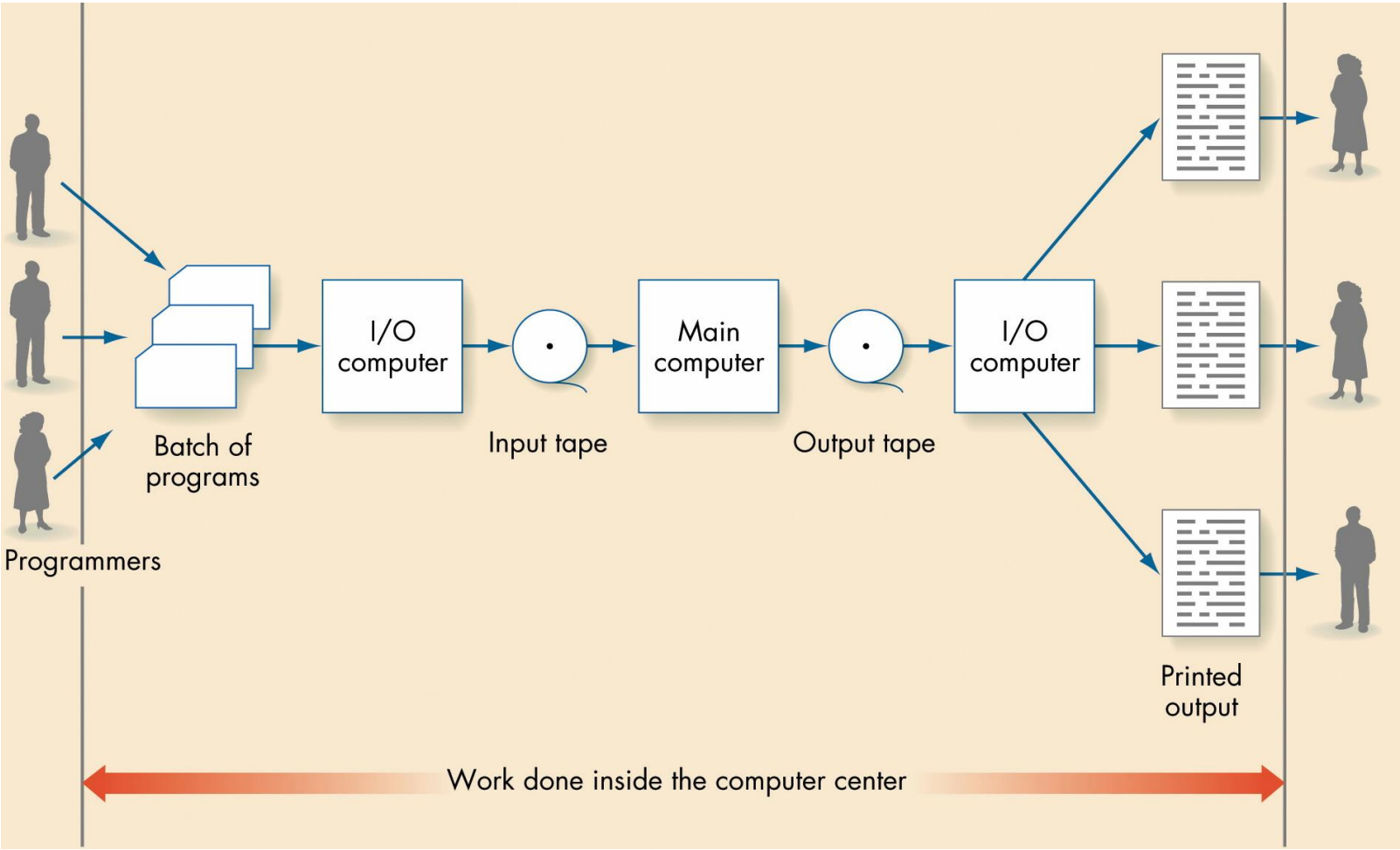
The Second Generation (1955-1965): Batch Systems

- **Birth of Operating Systems**
- Transistors replacing vacuum tubes ➔ computers became more ***faster***
- **Batch Processing Systems:**
  - Instead of running one job at a time with long setup periods
  - Jobs were collected into batches and processed sequentially without human intervention.

Batch processing



Batch processing



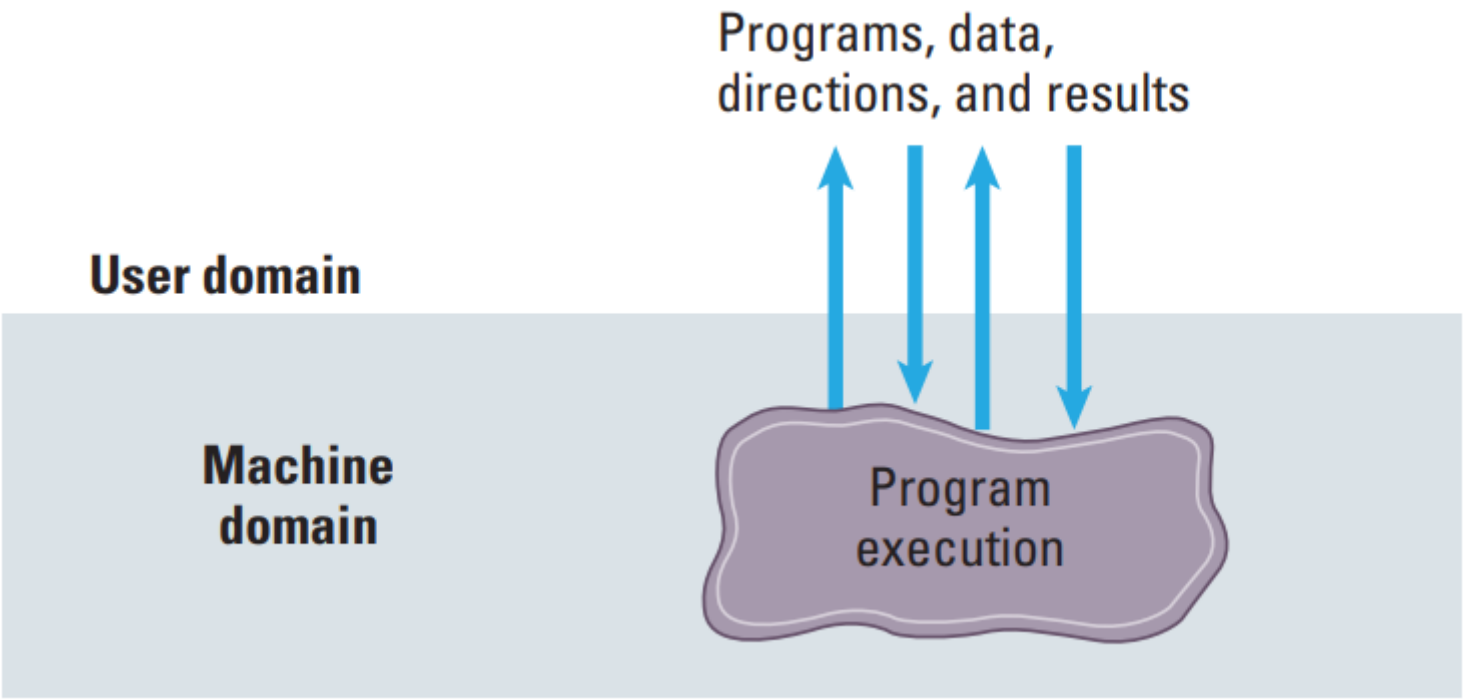
Batch processing - How it worked

- **Programmers** would write programs on punch cards
- **Operators** collected cards from many programmers
- **Cards** were batched together and loaded onto magnetic tape
- The computer read and **executed jobs** from tape sequentially
- **Output** went to another tape or printer

The Limitations

- **No interactivity**
  - Submit your job and wait hours or days for results
- **No multitasking**
  - CPU idle during I/O operations
- **Wasted resources**
  - While one program waited for slow I/O devices, CPU sat unused

Interactive processing



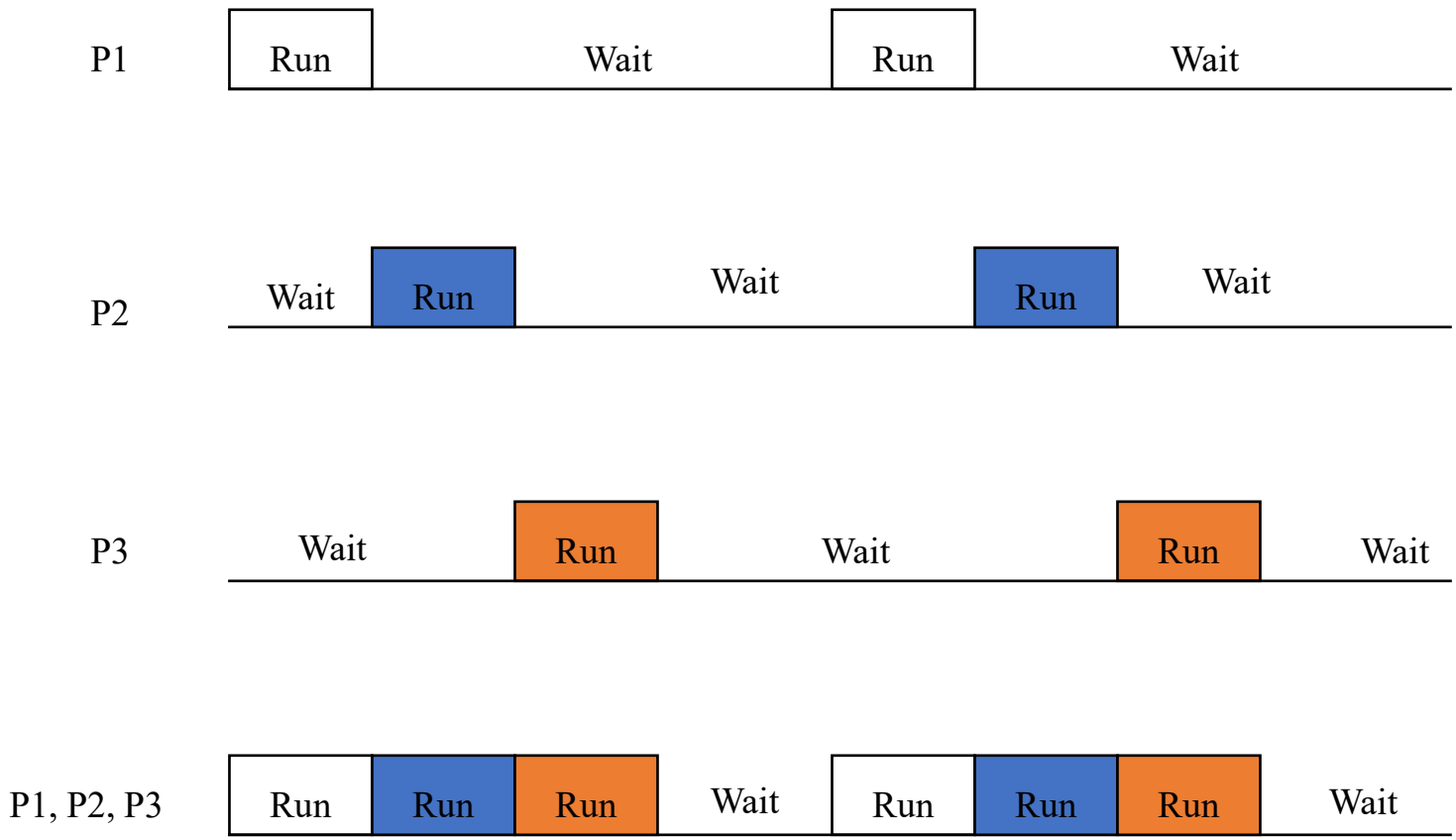
The Third Generation (1965-1980)

- **Multiprogramming and Time-Sharing**
- This era brought integrated circuits, making computers **smaller**, **faster**, and **cheaper**.
- More importantly, it introduced revolutionary OS concepts still used today.
  - Multi-programming
  - Time-Sharing Systems

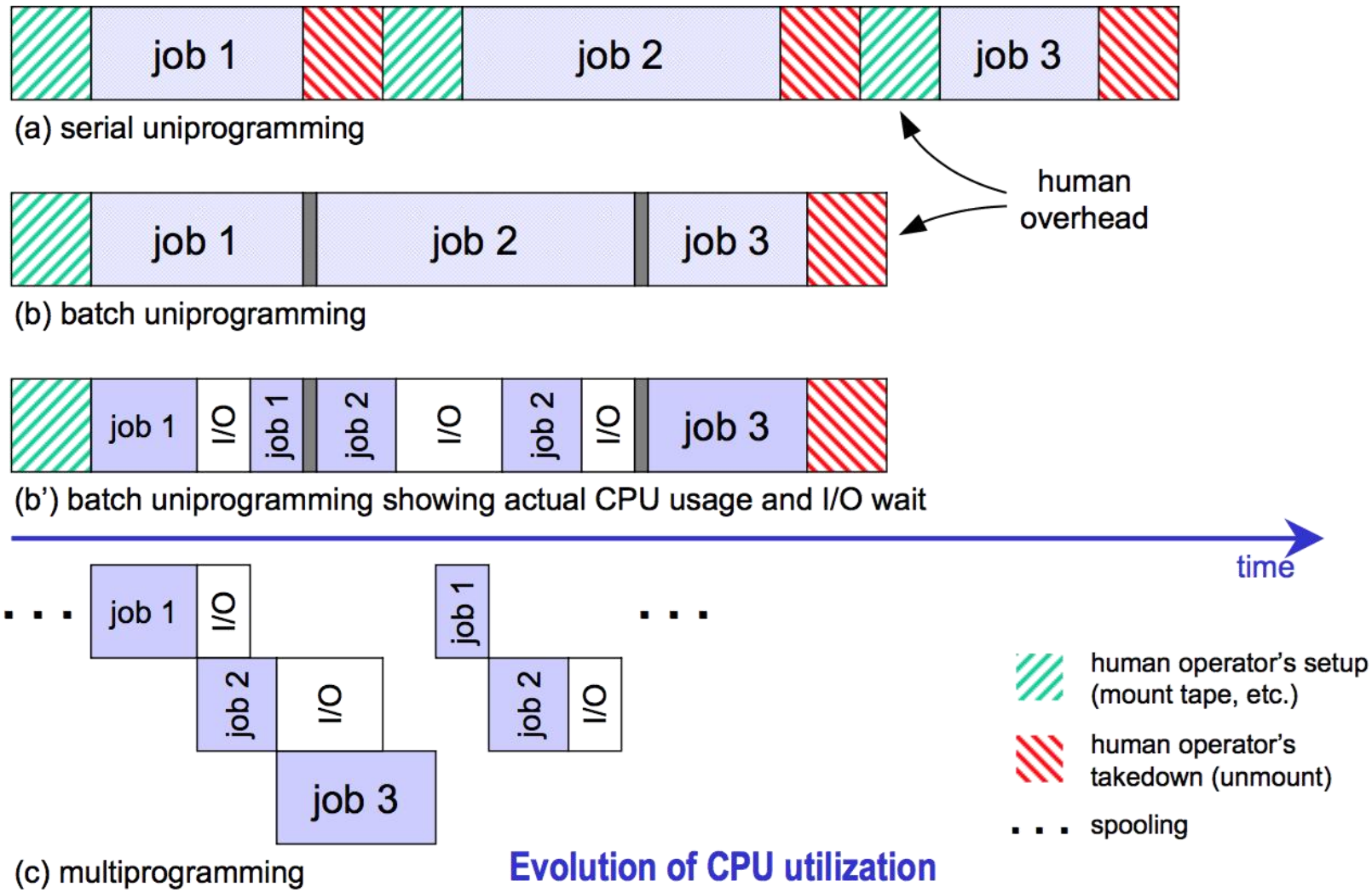
The Third Generation (1965-1980)

- **Multi-programming**
  - Keep multiple jobs in memory simultaneously.
  - When one job waits for I/O, the CPU switches to another job.
  - Result: Much better CPU utilization and throughput.
- **Time-Sharing Systems**
  - Allowed multiple users to interact with the computer simultaneously, each getting small time slices of CPU.

Multi tasking processing



Evolution of CPU Utilization



Evolution of CPU utilization

Multics (1969)

- The ***Multiplexed Information and Computing Service*** was an ambitious project by MIT, Bell Labs, and GE.
- Though commercially unsuccessful, Multics introduced groundbreaking concepts:
  - Hierarchical file system
  - Dynamic linking
  - Shared memory
  - Security rings (protection levels)
  - Virtual memory
- Many modern OS features originated in Multics.

Unix (1969-1980s)

- When Bell Labs withdrew from the Multics project, two researchers—**Ken Thompson** and **Dennis Ritchie**—decided to create a simpler system.
- Unix (1969) was born at AT&T Bell Labs, initially written in assembly language for the PDP-7 minicomputer.
- In 1973, Ritchie and Thompson rewrote Unix in C, a new programming language Ritchie had created.

Why Unix Was Revolutionary

- **Portability**: written in C (rather than assembly) made Unix portable to different hardware architectures
- **Simplicity**: Unix followed the philosophy of small, modular tools that do one thing well and can be combined.
- **Hierarchical File System**: Everything is a file—devices, processes, regular files all accessed through the same interface.
- **Pipes and Redirection**: Programs could be chained together, with output of one becoming input to another.
- **Multitasking** and **Multi-user**: Multiple users could work simultaneously.

Berkeley Unix (BSD, 1977)

- Unix is distributed to universities essentially for **free**, including source code.
- **Berkeley Unix (BSD**, 1977): UC Berkeley created their own version, adding features like:
  - TCP/IP networking
  - Virtual memory improvements
  - The vi editor
  - The C shell
- **The Unix Wars**: Different versions proliferated—System V (AT&T), BSD, Xenix, and others—leading to fragmentation but also innovation

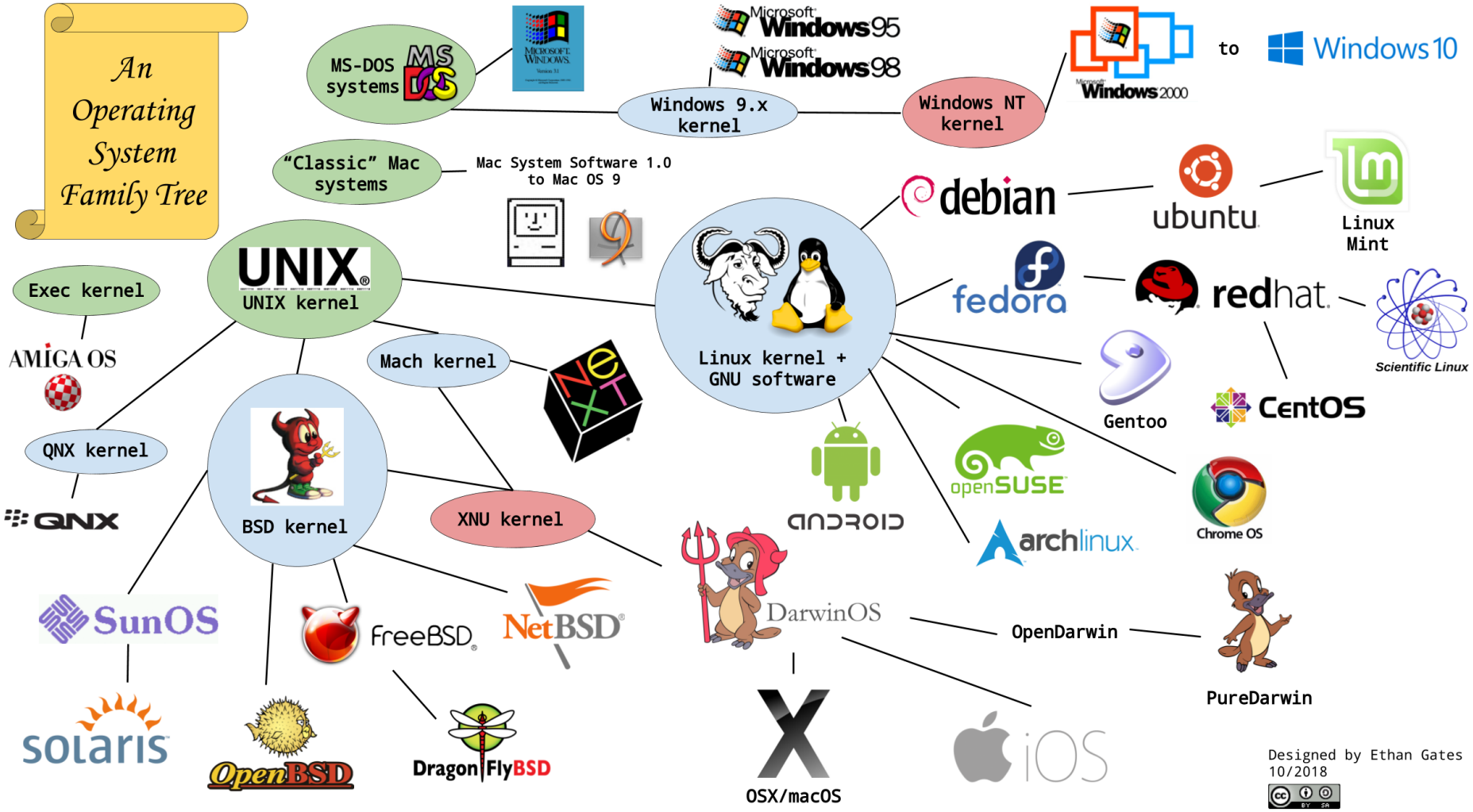
Personal Computer Revolution (1970s-1980s)

- MS-DOS (1981)
- Windows 1.0 (1985)
- Windows 3.0 (1990)
- Windows 95 (1995)
- Windows NT (1993) (Kernel)
- Windows 2000
- Windows XP (2001)
- Windows Vista (2007)
- Windows 7 (2009)
- Windows 10 (2015)
- Windows 11 (2021)

Linux: The Open Source Revolution

- Linux (1991): Finnish student Linus Torvalds created a **Unix-like kernel** as a hobby project and released it with open-source license.
- Linux Today:
  - Powers most web servers and cloud infrastructure
  - Runs supercomputers (100% of top 500)
  - Dominates mobile through Android
  - Increasingly used in embedded systems

OPERATING SYSTEMS



OPERATING SYSTEMS

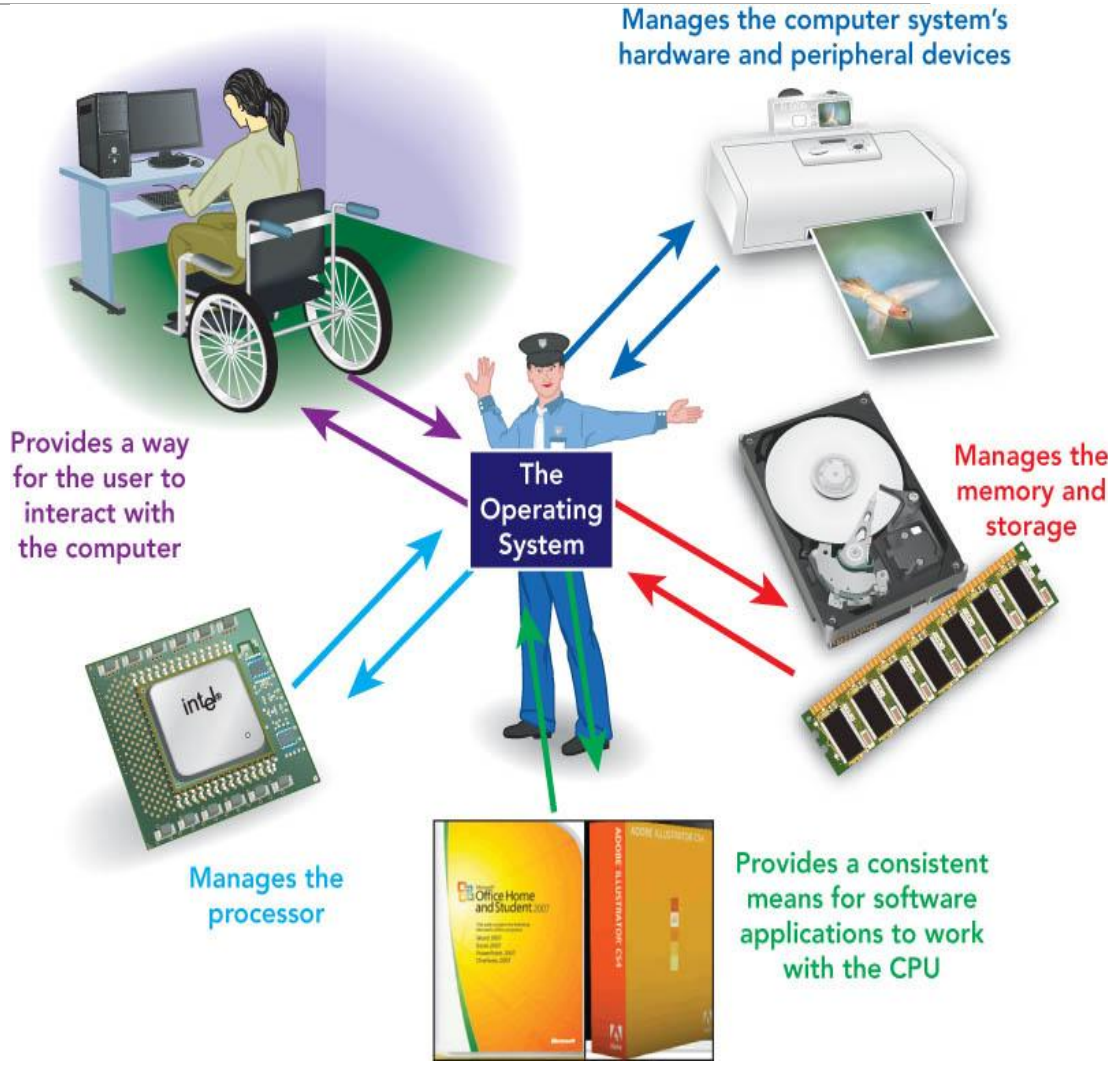
- **Smartphones**: Android (71.67%), and Apple's iOS (27.73%)
- **Desktop computers and laptops**: Microsoft Windows (71.47%), Apple's macOS (15.45%), desktop Linux 6.28% (including Google's ChromeOS at 1.73%). The remaining 6.81% is unknown
- **Tablets**: Apple's iPadOS (a variant of iOS) (56.01%) and Android (43.79%)
- Linux has completely dominated the **supercomputer** field since 2017, with all of the top 500 most powerful supercomputers in the world running a Linux distribution.
- Linux is also most used for **web servers**, and the most common Linux distribution is Ubuntu, followed by Debian.

* ***Data in Sept. 2024***

Generation	Approximate Dates	Major Advances
First	1945–1955	No operating system available Programmers operated the machine themselves
Second	1955–1965	Batch operating systems Improved system utilization Development of the first command language
Third	1965–1985	Multiprogrammed operating systems Time-sharing operating systems Increasing concern for protecting programs from damage by other programs Creation of privileged instructions and user instructions Interactive use of computers Increasing concern for security and access control First personal computer operating systems
Fourth	1985–present	Network operating systems Client-server computing Remote access to resources Graphical user interfaces Real-time operating systems Embedded systems
Fifth	??	Multimedia user interfaces Massively parallel operating systems Distributed computing environments

Basic functions of operating systems

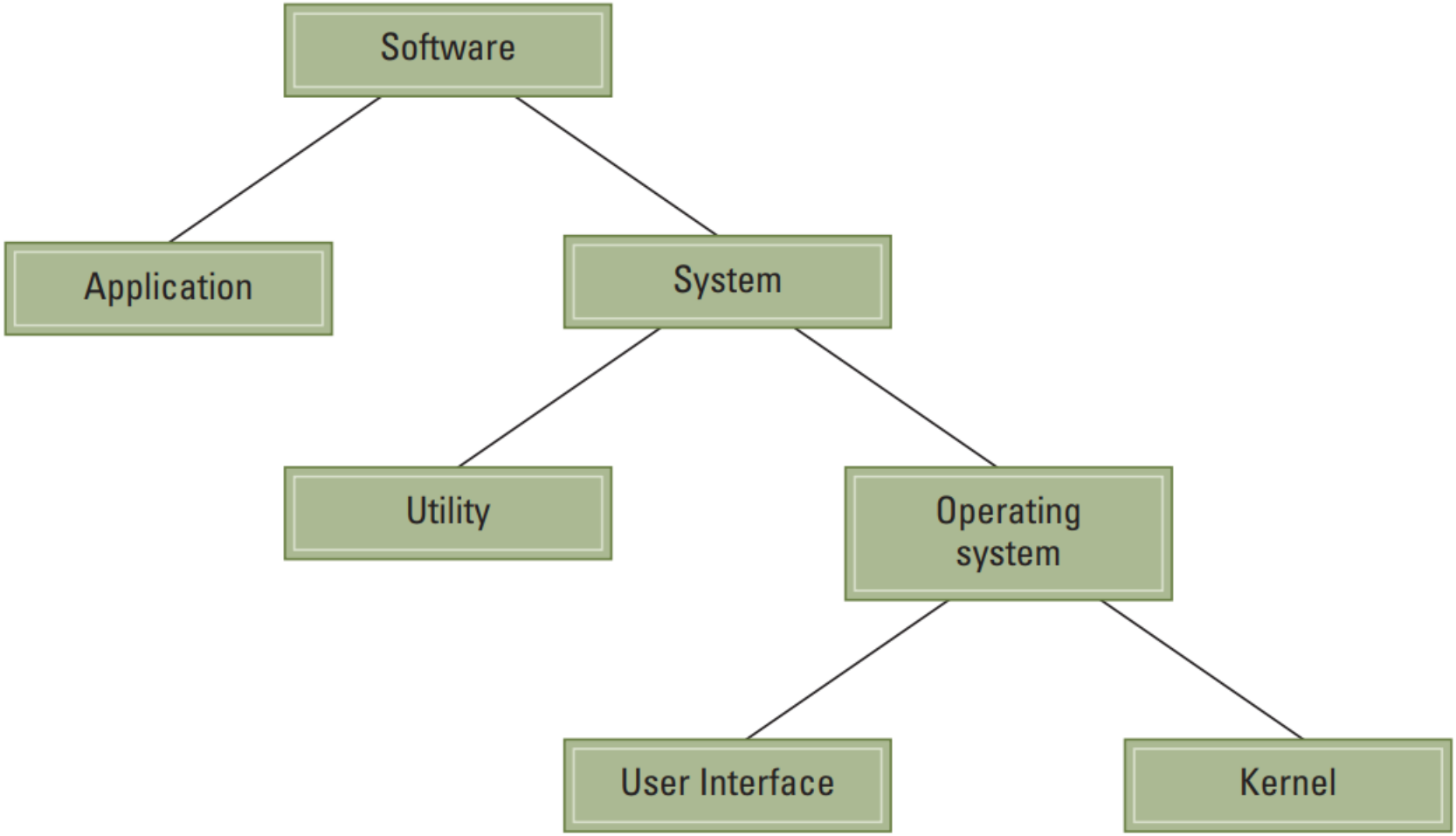
- Five basic functions
  - Starts the computer
  - Manages applications
  - Manages memory
  - Handles input and output device messages
  - Provides a user interface for communication



Types of Software

- Application software
  - Performs specific tasks for users
- System software
  - Provides infrastructure for application software
  - Consists of operating system and utility software

Software classification



System Software

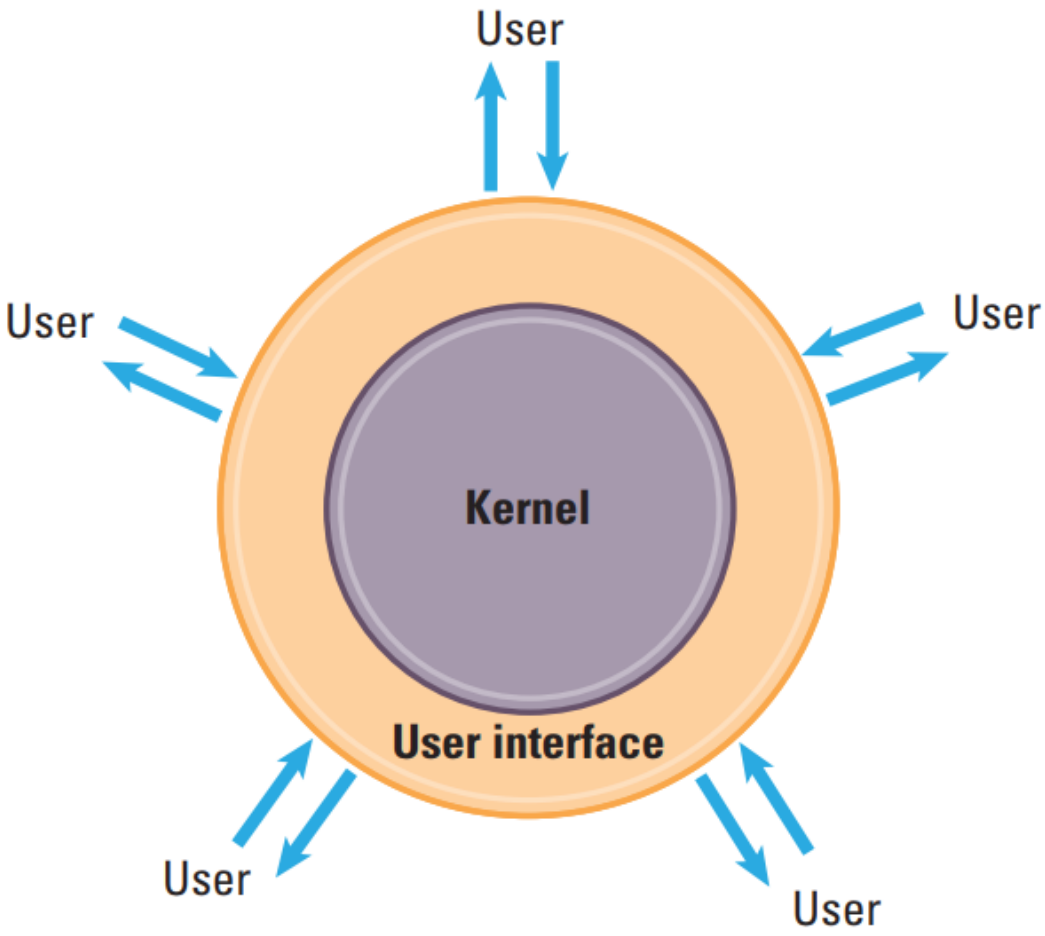
- Consists of all the programs that enable the computer and its peripheral devices to function smoothly
- Divided into two main categories:
  - The operating system
  - System utilities (utility programs)

Operating System Components

Operating System Components

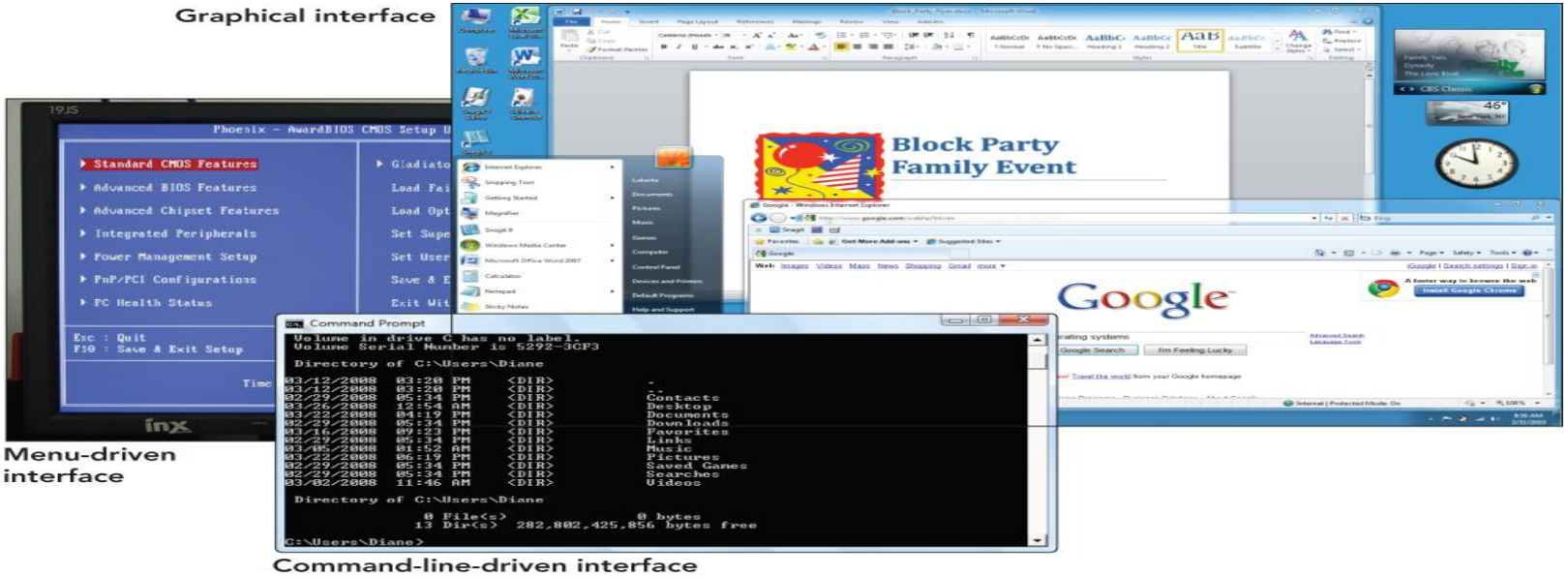
- **User Interface:** Communicates with users
  - Text based (Shell)
  - Graphical user interface (GUI)
- **Kernel:** Performs basic required functions
  - File manager
  - Device drivers
  - Memory manager
  - Scheduler and dispatcher

User - User interface - Kernel



User interface

- Types of user interfaces
  - Graphical user interface (GUI)
  - Menu-driven user interface
  - Command-line user interface



User interface

```
  Left        File        Command      Options      Right
/software
      Name          Size      MTime
/..                 4096  Oct  2 04:02
/ICAClient-3.0      2048  Jan  6  2003
/aida-2.1.1         2048  Apr 28  2003
/amber-6.0          2048  Feb 27  2004
/amber-7.0          2048  Mar  5  2004
/amber-7.0p         2048  Apr 16  2004
/amber-8            2048  Dec 22  2004
~ansys61              34  Jan  7  2003
~ansys71              34  Nov 28  2003
/ant-1.6            2048  Aug 10 13:26
/apache-1.3.27      2048  Dec 16  2002
/apache-1.3.28      2048  Jan  6  2004
/apache-1.3.33      2048  Feb  7  2005
/autoconf-2.57      2048  May 27  2004
/autodock-305       2048  Jan  5  2001

/ICAClient-3.0

/etc
      Name          Size      MTime
/..                 4096  Oct  2 04:02
/.java                30  May 13  2004
/ada                4096  Aug  9  2001
/conf                151  Jul 19  2000
/config             4096  Dec 13  2004
/cron.d              133  Sep 29 20:23
/default              75  Aug 12  2004
/dt                   27  Apr  5  2003
/fscklogs             39  Aug  3  2000
~fstyp.d              15  Apr 25  2000
~httpd                20  Jul 19  2000
/init.d             4096  Sep 21 15:45
/js                 4096  Aug  9  2001
/lost+found         4096  Oct  8  2004
/mail               4096  May  2 10:04

/cron.d

Hint: Keys not working in xterms? Use our xterm.ad, .ti and .tcap files.
aisa:/software>$                                                    [^]
1Help  2Menu  3View  4Edit  5Copy  6RenMov 7Mkdir  8Delete 9PullDn 10Quit
```

Kernel - File Manager

- Organizes and controls **how data is stored and retrieved** on disk
- **Directory** (or **Folder**): A user-created bundle of files and other directories (subdirectories)
- **Directory Path:** A sequence of directories within directories

Kernel - File Manager

- Searching for and managing files
  - File manager—utility software that organizes and manages data
    - Copy files
    - Determine how and where files are stored
    - Delete files
  - Search utility—enables you to locate files

Kernel - Memory Manager

- **Memory management:** Controls how RAM (physical memory) is allocated and used by programs.
- Main memory provides a fast storage that can be accessed by the CPU. For a program to be executed, it must in the main memory.

Kernel - Memory Manager

- An Operating System does the following activities for memory management:
  - **Keeps tracks** of main memory: what part of it are in use by whom, what part are not in use.
  - In **multiprogramming**, the OS decides which process will get memory when and how much.
  - **Allocates** the memory when a process requests it to do so.
  - **De-allocates** the memory when a process no longer needs it or has been terminated.

Kernel - Memory Manager

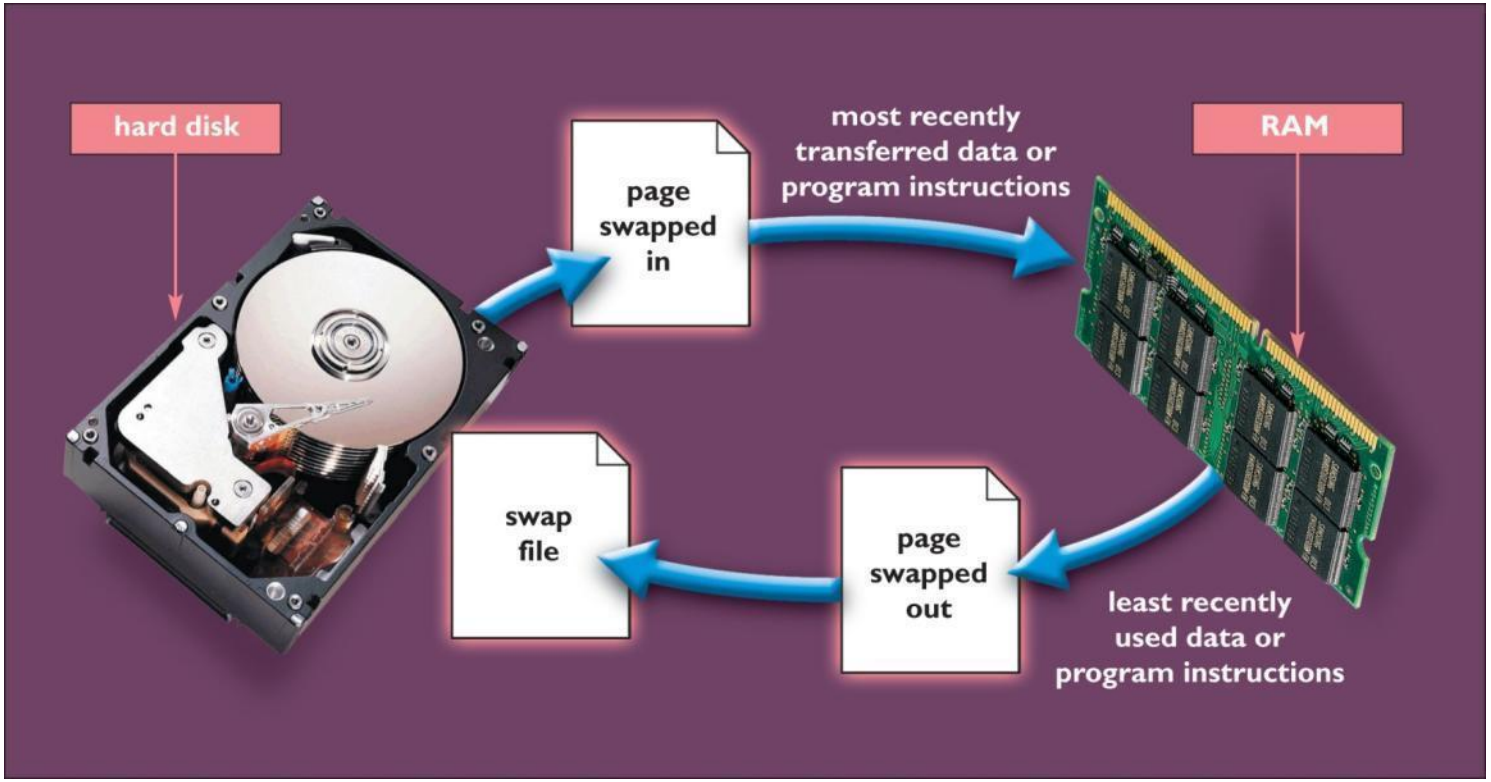
- Allocates space in main memory
- May create the illusion that the machine has more memory than it actually does (**virtual memory**) by playing a “shell game” in which blocks of data (**pages**) are shifted back and forth between main memory and mass storage

Kernel - Memory Manager

- Virtual memory—uses portion of hard disk to extend RAM
  - Pages—units of fixed size, contain program instructions and data
  - When RAM is full, copies of pages are temporarily stored in a swap file, a special hard disk file.
  - Transferring files between RAM and the hard disk—paging
  - Excessive paging—thrashing

- Adding more RAM—best way to improve computer performance:
  - Paging slows computer.
  - Accessing data from hard disk is slower than accessing from RAM.

Kernel - Memory Manager



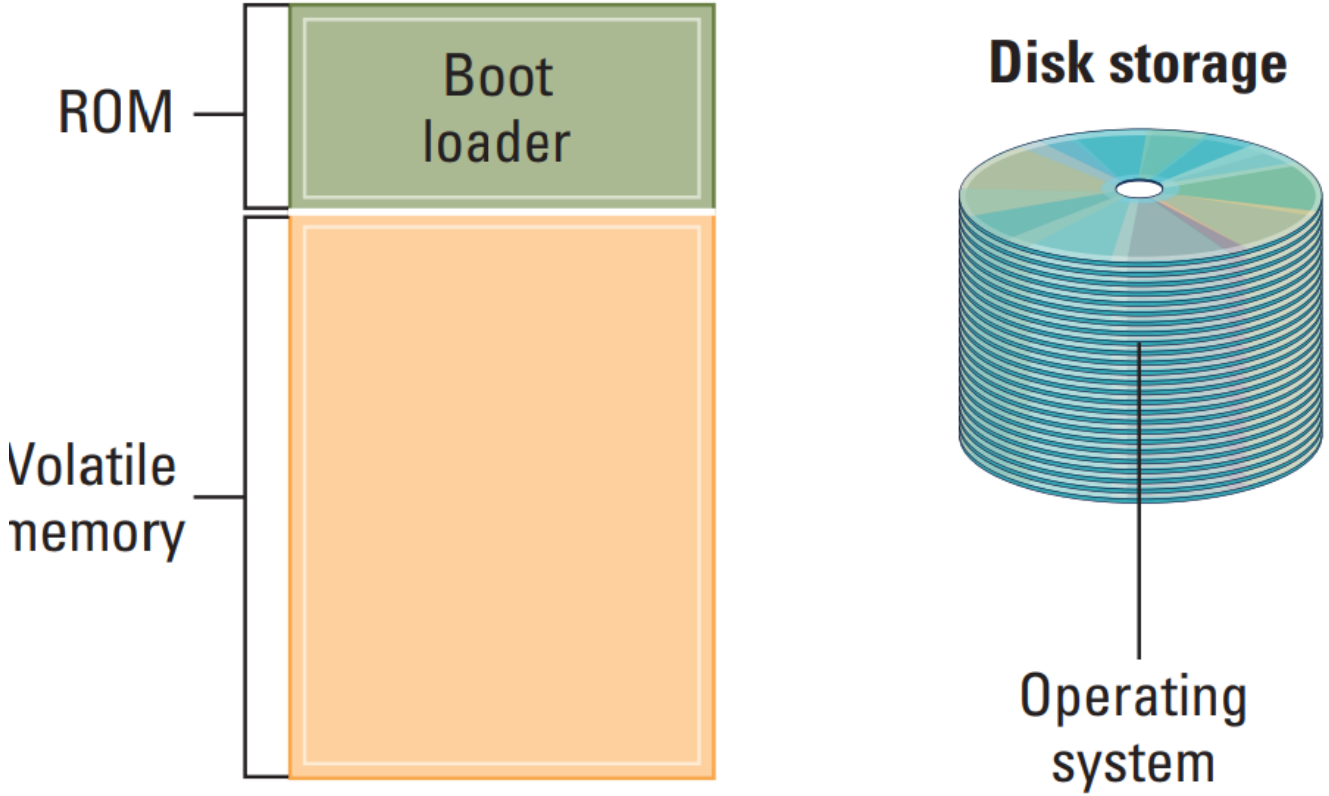
Getting It Started

Bootstrapping

- **Boot loader:** Program in ROM (example of firmware)
  - Run by the CPU when power is turned on
  - Transfers operating system from mass storage to main memory
  - Executes jump to operating system

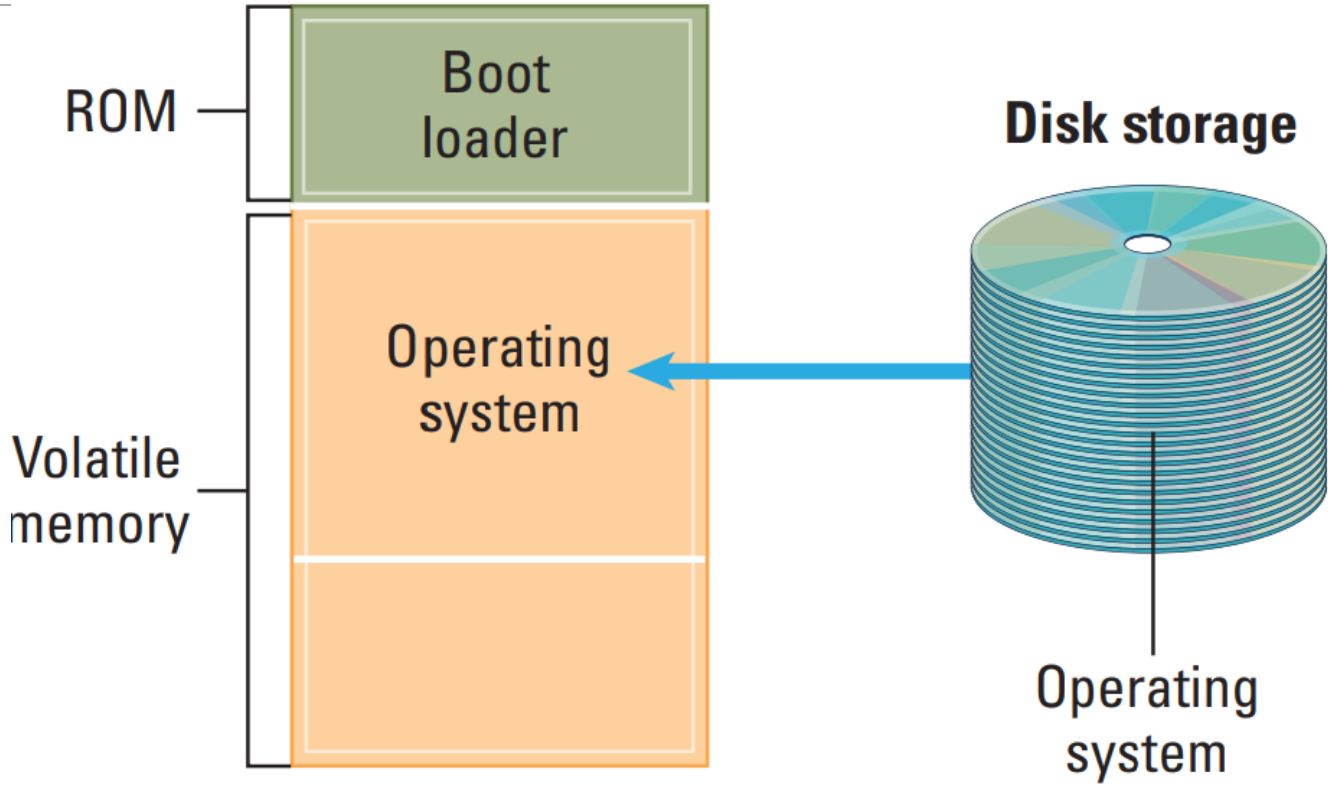
  - Cold boot: Starting computer when it has not yet been turned on
  - Warm boot: Restarting a computer that is already on

The booting process



Step 1: Machine starts by executing the boot loader program already in memory. Operating system is stored in mass storage.

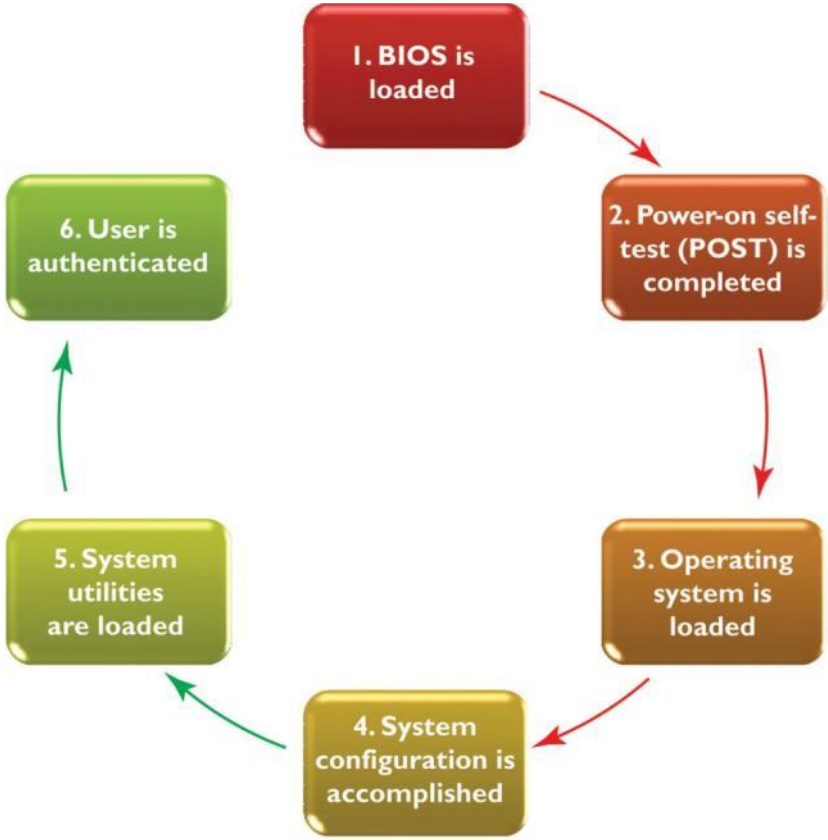
The booting process



Step 2: Boot loader program directs the transfer of the operating system into main memory and then transfers control to it.

The booting process

The Six Steps of Booting a System



Step 1: Activate the BIOS and Setup Program

- **Power supply** sends electricity to all components
- **CPU** receives power and automatically resets to a predefined state
- CPU's **instruction pointer** is hardwired to jump to a fixed memory address in **ROM**
- This **ROM** address contains the **BIOS/UEFI firmware**
- **BIOS code** begins executing from ROM chip on the motherboard

BIOS

```
CMOS Setup Utility - Copyright (C) 1984-1999 Award Software

▶ Standard CMOS Features          ▶ Frequency/Voltage Control
▶ Advanced BIOS Features            Load Fail-Safe Defaults
▶ Advanced Chipset Features         Load Optimized Defaults
▶ Integrated Peripherials           Set Supervisor Password
▶ Power Management Setup            Set User Password
▶ PnP/PCI Configurations            Save & Exit Setup
▶ PC Health Status                  Exit Without Saving

Esc : Quit                          ↑ ↓ → ←   : Select Item
F10 : Save & Exit Setup

                    Time, Date, Hard Disk Type...
```

Step 1: Activate the BIOS and Setup Program

- **Definition**: BIOS
  - **Firmware** that initializes and tests hardware during boot
  - Provides **runtime services** for OS and programs
  - Acts as an **intermediary** between the OS and hardware.
- **Core purpose**:
  - Provides the computer with low-level control
  - Descriptions of internal hardware components
  - Enabling the system to start and communicate with devices before the OS loads.
- If you don't understand a BIOS setting, **don't change it**. Research first, change cautiously, and always have a recovery plan

Step 1: Activate the BIOS and Setup Program

- BIOS Settings
  - Change the Boot Order
  - Load BIOS Setup Defaults
  - Remove a BIOS Password
  - Create a BIOS Password
  - Change the Date and Time
  - Change Floppy Drive Settings
  - Change Hard Drive Settings
  - Change CD/DVD/BD Drive Settings
  - View Amount of Memory Installed
  - Change the Boot Up NumLock Status
  - Enable or Disable the Computer Logo
  - Enable or Disable the Quick Power On Self Test (POST)

Step 2: Initiate the Power-On Self-Test

- **Definition**: Power-On Self-Test (POST)
  - Is a diagnostic testing sequence
  - Eun by the BIOS/UEFI firmware
  - Immediately after the computer is powered on, before any OS loads.
- **Purpose**:
  - Verify that all essential hardware components are present
  - Properly connected
  - Functioning correctly before attempting to boot the OS.
- **When it runs**:
  - Every time the computer do a cold boot
  - During restart (warm boot), though may be abbreviated

Step 2: Initiate the Power-On Self-Test

Hardware Components Checked:

- **CPU:** System cannot function without working CPU
- **RAM**
- **Video/Graphics Card:** if not working, you won't see error messages (rely on beep codes)
- **Motherboard Chipset**
- **Keyboard:** Some systems show "Keyboard Error" if none detected
- **Storage Devices**
- **Other Peripherals**
  - USB controllers
  - Network adapters (onboard)
  - Sound card (basic detection)

Step 2: Initiate the Power-On Self-Test

Normal POST Sequence (Success)

1. Power LED lights up
2. Fans start spinning (CPU, case, GPU)
3. Manufacturer logo appears on screen (Dell, HP, ASUS, Lenovo, etc.)
4. Memory count may scroll by (optional, can be disabled)
5. Brief hardware summary may appear (CPU, RAM, drives)
6. Single short beep (on most systems) = POST passed
7. Boot sequence continues to bootloader

Step 2: Initiate the Power-On Self-Test

- When POST Fails:
  - POST uses multiple methods to report errors because different failures affect what's available
- Beep Codes (Audio Feedback): long, short, continuous
- No Beeps at All
- On-Screen Error Messages
- **LED Indicators** (Visual on Case/Motherboard)

The booting process

- **Step 3: Load the Operating System**
  - BIOS
    - Looks for the operating system
    - Loads the kernel into memory—the central part of the operating system
  - The operating system loads the system configuration information.

The booting process

- **Step 4: Configure the System**
  - Operating system
    - Checks the registry
      - Database that stores information about software and peripherals choices, for configuration information
    - Checks the configuration for drivers
      - Utility programs containing instructions for the proper functioning of peripheral devices.
    - Automatically detects plug-and-play (PnP) devices
    - Checks for conflicts between devices
    - Installs and loads needed drivers

The booting process

- **Step 5: Load System Utilities**
  - Antivirus software
  - Speaker volume control
  - Power management options

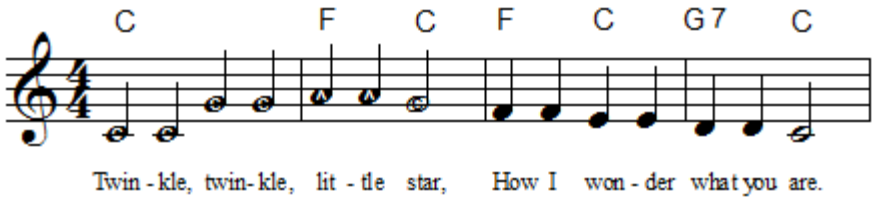
The booting process

- **Step 6: Authenticate a User**
  - Verifies authorized users
    - Enter an authentication/login user name and password
  - Profile—a record of a specific user's preferences for the desktop theme, icons, and menu styles
  - Account—for multiuser computer systems each user has an account
    - Consists of user name, password, and storage space
    - Created by server/computer administrator

Coordinating machine’s activities

Processes

- **A process is a program in execution**
- **Key distinction**:
  - **Program**: Passive entity - executable file stored on disk
  - **Process**: Active entity - program loaded into memory and being executed by CPU (dynamic, living execution)



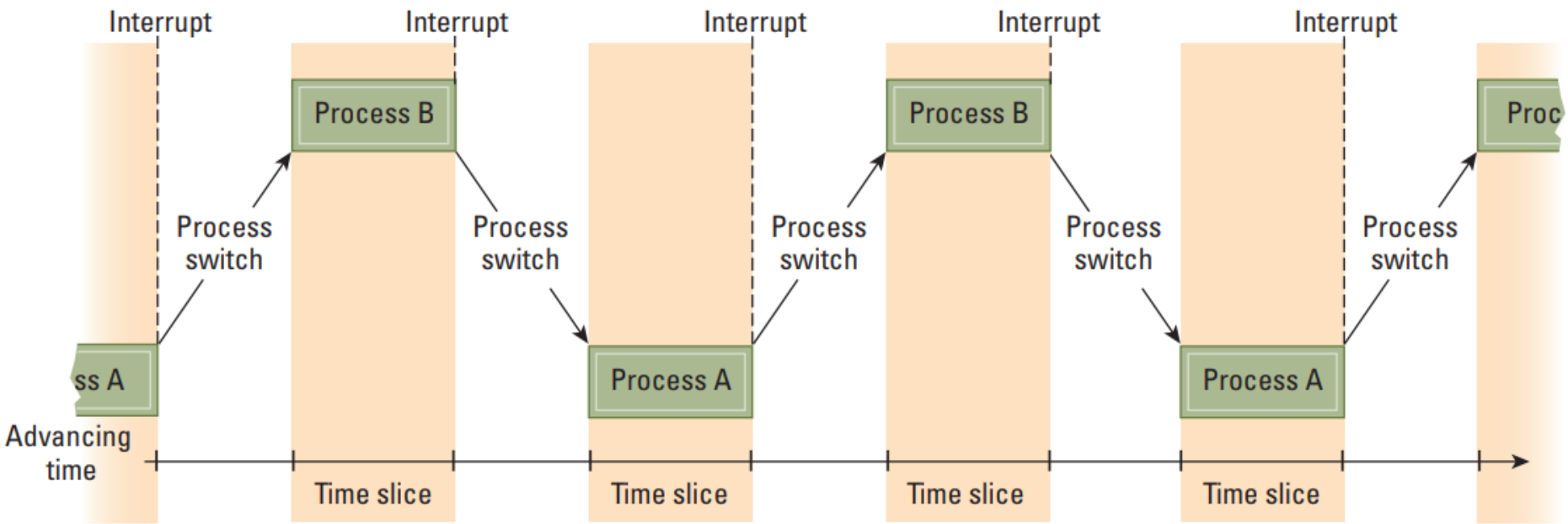
Processes

- **Multiple processes from one program**
  - Opening Microsoft Word twice creates **two separate processes**
  - Each has independent memory, state, and execution
  - Same code, but different **data** and **execution contexts**
- Process state: current status of the activity
  - Program counter
  - General purpose registers
  - Related portion of main memory

Process Administration

- **Scheduler:** Adds new processes to the process table and removes completed processes from the process table
  - **Process table**: a block of information in main memory to keep track of all the processes
- **Dispatcher:** Controls the allocation of time slices to the processes in the process table
  - The end of a time slice is signaled by an interrupt.

Time-sharing



Handling Competition for Resources

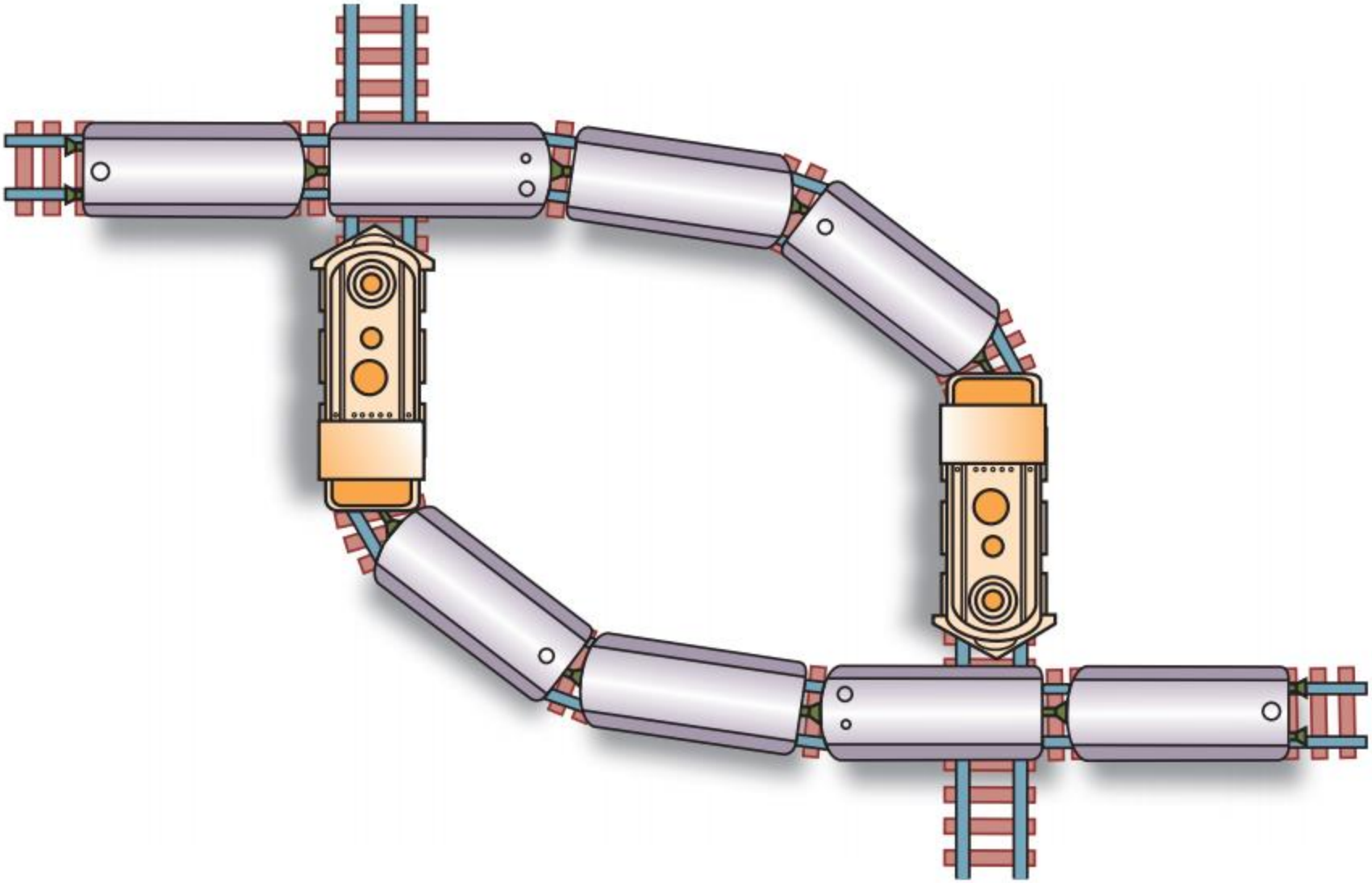
Handling Competition for Resources

- **Semaphore:** A “control flag”
- **Critical Region:** A group of instructions that should be executed by only one process at a time
- **Mutual exclusion:** Requirement for proper implementation of a critical region

Deadlock

- Processes block each other from continuing
- Conditions required for deadlock
  1. Competition for non-sharable resources
     - Converting nonshareable resources into shareable ones: spooling
  2. Resources requested on a partial basis
  3. An allocated resource can not be forcibly retrieved
     - Forcibly retrieving some of the allocated resources

A deadlock resulting from competition

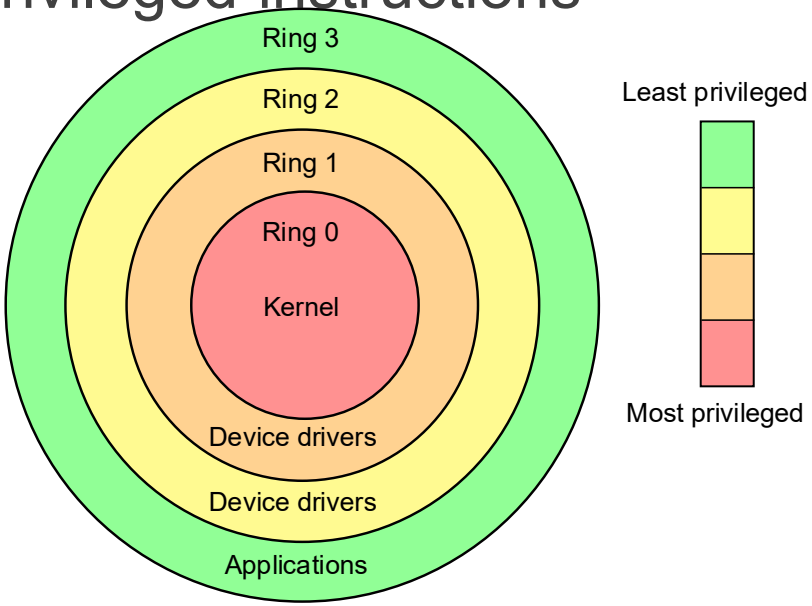


Security

- Attacks from outside: Login
  - Problems
    - Insecure passwords
      - password cracker, network sniffer, Trojan horse login
    - Sniffing software
      - spyware, sniffing software
  - Counter measures
    - Auditing software

Security

- Attacks from within
  - Problem: Unruly processes
  - Counter measures: Control process activities via privileged modes and privileged instructions



Questions and Answers

Quiz

Which OS is not designed for smartphones and PDAs?

A. Android

B. Mac OS

C. Symbian

D. Windows Mobile

Quiz

What utility program reduces a file size by as much as 80 percent by substituting short codes for lengthy data patterns?

A. Defragmentation

B. Compression

C. Interrupt

D. Cleanup

Quiz

Which of the following is an OS function?

A. Creating letters

B. Defragmenting a disk

C. Managing memory

D. Writing an e-mail

Quiz

Virtual memory is used when:

A. booting fails.

B. an IRQ conflict occurs.

C. RAM is full.

D. a power-on-self-test fails.

Quiz

Which test makes sure the computer and its peripherals are working correctly during the start-up process?

A. BIOS

B. Upgrade

C. ReadyBoost

D. POST

Quiz

Which of following are functions of the operating system?

A. File management

B. CPU management

C. Task management

D. Booting

E. All of these

Quiz

Which of the following refers are specialized software programs that allow input and output devices to communicate with the rest of the computer system?

A. Multitasking

B. Boot-disks

C. Utility programs

D. Device drivers

E. Service packs

Quiz

Which is the common consumer computer interface used today?

A. command-line interface

B. graphical user interface

C. menu-driven interface

D. electronic user interface

E. biometric user interface